

BIG DATA ANALYTICS

Comprehensive Viva Preparation Guide

SPIT | All 10 Experiments Covered

Hadoop & HDFS MapReduce

HBase

Hive

Pig

Spark

Pandas / EDMemory-Efficient

1. Big Data — The Foundation

What is Big Data?

Big Data refers to extremely large datasets that cannot be processed or stored by traditional database management systems within a reasonable time. It is characterised by the **5 V's**:

V	Name	Description
Volume	Volume	Petabytes/exabytes of data generated every second
Velocity	Velocity	Speed at which data is generated and processed (real-time streams)
Variety	Variety	Structured (SQL), semi-structured (JSON/XML), unstructured (video, text)
Veracity	Veracity	Trustworthiness and quality of data
Value	Value	Business intelligence extracted from the data

Memory-Efficient Data Structures (Exp 1)

When dealing with big data, traditional in-memory structures become infeasible. Memory-efficient (probabilistic) data structures trade **exact answers for space savings**.

Structure	Use Case	Key Property
Bloom Filter	Set membership testing (e.g. URL seen?)	False positives possible, never false negatives. O(1) lookup
Count-Min Sketch	Frequency estimation of stream elements	Approximate counts with bounded error
HyperLogLog	Cardinality estimation (unique count)	~2% error using only ~1.5 KB for billions of items
Skip List	Ordered key-value store (used in LevelDB)	Probabilistic balanced tree, O(log n) avg ops
LSM Tree	Write-heavy storage (HBase, Cassandra)	Sequential writes, compaction in background
Trie / PATRICIA	String prefix lookups and autocomplete	O(length of key) lookup, space-efficient for common prefixes
Roaring Bitmaps	Compressed bitsets for large integer sets	Faster set operations than plain bitsets

2. Hadoop & HDFS (Exp 2)

Hadoop Ecosystem Overview

Apache Hadoop is an open-source framework for distributed storage and processing of large datasets across clusters of commodity hardware. Core components:

- **HDFS** (Hadoop Distributed File System) — distributed storage layer
- **YARN** (Yet Another Resource Negotiator) — cluster resource management
- **MapReduce** — distributed batch processing engine
- **Common** — shared utilities and libraries

HDFS Architecture

HDFS follows a **master-slave architecture**. Files are split into fixed-size **blocks (default 128 MB)** and replicated across DataNodes for fault tolerance.

Component	Role
NameNode (Master)	Manages filesystem metadata (file names, block locations, permissions). Stores in FSImage + EditLog. SPOC for the cluster.
DataNode (Slave)	Stores actual data blocks. Sends heartbeats to NameNode every 3 seconds. Reports block list every hour.
Secondary NameNode	NOT a backup NameNode. Merges FSImage + EditLog periodically to prevent EditLog from growing too large.
Block	Default 128 MB. Each block replicated 3 times across different racks. Replication factor is configurable.
Rack Awareness	HDFS places replicas on different racks to survive rack-level failures.

Key HDFS Commands

Command	Description
hdfs dfs -ls /	List root directory
hdfs dfs -mkdir /user/student	Create directory
hdfs dfs -put localfile.txt /hdfs/path	Upload file to HDFS
hdfs dfs -get /hdfs/path localfile.txt	Download file from HDFS
hdfs dfs -cat /hdfs/file.txt	View file contents
hdfs dfs -rm -r /path	Delete file/directory recursively
hdfs dfs -mv /src /dest	Move/rename
hdfs dfs -cp /src /dest	Copy
hdfs dfs -du -s /path	Show disk usage
hdfs dfs -chmod 755 /path	Change permissions
hdfs fsck /	Check filesystem health
hdfs dfsadmin -report	Cluster status report
hdfs dfs -setrep -w 2 /file	Change replication factor

3. Pandas & Exploratory Data Analysis (Exp 3)

What is EDA?

Exploratory Data Analysis (EDA) is the process of **visually and statistically summarising** a dataset to discover patterns, spot anomalies, test hypotheses, and check assumptions before formal modelling.

Key Pandas Operations for EDA

- **df.head(n)** — first n rows; **df.tail(n)** — last n rows
- **df.info()** — dtypes, non-null counts, memory usage
- **df.describe()** — count, mean, std, min, quartiles, max for numeric columns
- **df.isnull().sum()** — count missing values per column
- **df.value_counts()** — frequency of unique values
- **df.corr()** — Pearson correlation matrix
- **df.groupby('col').agg({'col2': 'mean'})** — grouped aggregation
- **df.drop_duplicates()** — remove duplicate rows
- **df['col'].fillna(df['col'].mean())** — impute missing values
- **pd.get_dummies(df, columns=['cat_col'])** — one-hot encoding

EDA Steps (Standard Flow)

Step	Action	Tool
1	Load data	pd.read_csv(), pd.read_json()
2	Shape inspection	df.shape, df.columns, df.dtypes
3	Missing value analysis	df.isnull().sum(), heatmap
4	Univariate analysis	histograms, boxplots, countplots
5	Bivariate analysis	scatter plots, correlation, crosstab
6	Outlier detection	IQR method, Z-score, boxplots
7	Feature engineering	encoding, scaling, binning
8	Insights & conclusions	Summarise findings

4. MapReduce — Distributed Processing (Exp 4, 5, 6)

MapReduce Programming Model

MapReduce is a programming model for processing large datasets in parallel across a distributed cluster. Inspired by functional programming's map and reduce operations. Introduced by Google in 2004, implemented in Hadoop.

Phases of MapReduce

Phase	Description
Input Split	Input data divided into fixed-size chunks called InputSplits. One Mapper per split.
Map	User-defined map(key, value) function. Emits intermediate (key, value) pairs.
Shuffle	Framework groups all values with the same intermediate key together. Network-intensive.
Sort	Intermediate keys sorted within each Reducer. Happens automatically.
Reduce	User-defined reduce(key, [values]) function. Produces final output.
Output	Final key-value pairs written to HDFS.

Word Count Example — The Hello World of MapReduce

```
# MAPPER (mapper.py) import sys for line in sys.stdin: for word in line.strip().split():
print(f'{word.lower()}\t1') # REDUCER (reducer.py) import sys current_word, total = None,
0 for line in sys.stdin: word, count = line.strip().split('\t') if word == current_word:
total += int(count) else: if current_word: print(f'{current_word}\t{total}') current_word,
total = word, int(count) if current_word: print(f'{current_word}\t{total}')
```

Combiner — Mini-Reducer

A **Combiner** is an optional local aggregation step that runs on the Mapper's output before data is sent over the network. It reduces shuffle traffic significantly. For WordCount, the Combiner is identical to the Reducer. Not applicable when the operation is not commutative and associative (e.g. average).

MapReduce on Relational Algebra (Exp 5)

Operation	MapReduce Strategy
Selection (σ)	Mapper filters rows matching the predicate. Reducer is identity.
Projection (π)	Mapper extracts specified columns and removes duplicates.
Union	Both tables mapped; reducer discards duplicates.
Intersection	Tag tuples by source; reducer keeps tuples present from both sources.
Join (natural)	Map: emit (join_key, (table_tag, row)). Reduce: combine pairs with same key.
Aggregation (γ)	Map: emit (groupby_key, agg_value). Reduce: compute final aggregate.

5. Apache HBase — NoSQL on HDFS (Exp 7)

HBase Overview

Apache HBase is a **column-oriented, distributed, non-relational database** built on top of HDFS. It is modelled after Google's Bigtable. Ideal for sparse, large datasets with billions of rows and millions of columns.

- Provides **random, real-time read/write access** to data in HDFS
- Data organised into **Tables** → **Rows** → **Column Families** → **Columns** → **Versions**
- Each row identified by a unique **RowKey** (lexicographically sorted)
- Supports **automatic sharding** (splitting) of tables across RegionServers
- **Write-ahead log (WAL)** ensures durability
- Uses **LSM Tree** internally (MemStore + HFile)

HBase Data Model

Concept	Description
Table	Collection of rows. Similar to RDBMS table but sparse.
Row Key	Unique identifier for each row. Rows stored in sorted order of RowKey.
Column Family	Group of columns defined at table creation. Stored together on disk.
Column Qualifier	Column within a family. Created dynamically, no schema needed.
Cell	Intersection of row + column. Stores value + timestamp (version).
Timestamp	Version of each cell. Multiple versions of same cell can be stored.
Region	Contiguous range of rows. Unit of distribution across RegionServers.

HBase Shell — CRUD Commands

```
# Connect hbase shell # CREATE table with column family create 'students', 'personal', 'academic' # INSERT (Put) put 'students', 'row1', 'personal:name', 'Alice' put 'students', 'row1', 'personal:age', '20' put 'students', 'row1', 'academic:grade', 'A' # READ (Get specific row) get 'students', 'row1' # READ (Scan entire table) scan 'students' # UPDATE (same as put – overwrites cell) put 'students', 'row1', 'academic:grade', 'A+' # DELETE specific cell delete 'students', 'row1', 'personal:age' # DELETE entire row deleteall 'students', 'row1' # List tables list # Describe schema describe 'students' # Disable and drop table disable 'students' drop 'students'
```

6. Apache Hive — SQL on Hadoop (Exp 8)

Hive Overview

Apache Hive is a **data warehousing tool** built on top of Hadoop. It provides a SQL-like interface called **HiveQL (HQL)** to query data stored in HDFS. Hive translates HQL queries into MapReduce or Tez/Spark jobs internally.

- **Schema-on-read**: schema applied when data is read, not when written
- Data stored in HDFS as text, ORC, Parquet, or Avro files
- **Metastore**: relational database (Derby/MySQL) storing table definitions
- Supports **partitioning** and **bucketing** for performance
- Not suitable for OLTP; optimised for **batch analytics (OLAP)**
- Hive on Tez is ~100x faster than Hive on MapReduce

Key HiveQL Commands

```
-- Create database CREATE DATABASE retail_db; USE retail_db; -- Create external table
(data stays in HDFS) CREATE EXTERNAL TABLE sales ( order_id INT, customer STRING, amount
DOUBLE, order_date STRING ) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' LOCATION
'/user/hive/sales/'; -- Load data LOAD DATA INPATH '/hdfs/sales.csv' INTO TABLE sales; --
Basic queries SELECT customer, SUM(amount) AS total FROM sales GROUP BY customer ORDER BY
total DESC LIMIT 10; -- Partitioned table (improves query speed) CREATE TABLE logs
(user_id INT, event STRING) PARTITIONED BY (log_date STRING); -- Bucketed table CREATE
TABLE bucketed_sales CLUSTERED BY (customer) INTO 4 BUCKETS;
```

Internal vs External Tables

Internal (Managed) Table	External Table
Hive owns data AND metadata	Hive owns metadata only
DROP TABLE deletes data from HDFS	DROP TABLE deletes only metadata; data remains in HDFS
Default when no EXTERNAL keyword	Requires EXTERNAL keyword and LOCATION clause
Use when Hive is sole owner	Use when data shared across tools (HBase, Spark, etc.)

7. Apache Pig — Dataflow Processing (Exp 9)

Pig Overview

Apache Pig is a **high-level dataflow scripting platform** for Hadoop. It uses a language called **Pig Latin**, which describes a sequence of data transformations. Pig compiles Pig Latin into MapReduce jobs.

- **Schema-optional**: can process data without a schema (self-describing)
- Best for **ETL pipelines, log processing, sessionization**
- Pig Latin is **procedural** (step-by-step), unlike Hive's declarative SQL
- **UDFs (User Defined Functions)** can be written in Java, Python, JavaScript
- Handles both structured and unstructured data

Pig Latin — Core Operators

Operator	Description	Example
LOAD	Load data from HDFS	A = LOAD '/data/users.csv' USING PigStorage(',') AS (id:int, name:chararray,
FILTER	Filter rows	B = FILTER A BY age > 25;
FOREACH	Transform columns	C = FOREACH A GENERATE name, age*2 AS double_age;
GROUP	Group by key	D = GROUP A BY name;
ORDER	Sort data	E = ORDER A BY age DESC;
LIMIT	Take top N rows	F = LIMIT E 10;
JOIN	Join two relations	G = JOIN A BY id, B BY user_id;
UNION	Merge relations	H = UNION A, B;
DISTINCT	Remove duplicates	I = DISTINCT A;
STORE	Write to HDFS	STORE E INTO '/output/sorted' USING PigStorage(',');
DUMP	Print to console	DUMP E;
DESCRIBE	Show schema	DESCRIBE A;

Sessionization with Pig

Sessionization = grouping a user's events into sessions where a session gap > threshold (e.g. 30 min). Pig is excellent for this because it can LOAD raw logs, FILTER by user, ORDER by timestamp, and FOREACH with custom UDFs to detect session boundaries.

8. Apache Spark (Exp 10)

Spark Overview

Apache Spark is a **unified in-memory distributed computing engine**. It is up to **100x faster than Hadoop MapReduce** for in-memory processing and 10x faster on disk, because it avoids repeated disk I/O by caching intermediate results in RAM.

Spark vs Hadoop MapReduce

Hadoop MapReduce	Apache Spark
Disk-based (writes to HDFS after each step)	In-memory (keeps data in RAM between stages)
Slow for iterative algorithms (ML, graph)	Excellent for iterative workloads
Only batch processing	Batch + Streaming + ML + Graph
Java-centric API	Scala, Python (PySpark), R, Java
2-phase model (Map + Reduce)	DAG (Directed Acyclic Graph) execution plan
No native streaming	Spark Streaming, Structured Streaming

Spark Core Concepts

Concept	Description
RDD	Resilient Distributed Dataset. Immutable, distributed collection. Fault-tolerant via lineage graph.
DataFrame	Distributed table with named columns. Built on RDDs. Optimised via Catalyst and Tungsten.
Dataset	Type-safe, JVM-typed DataFrame. Available in Scala/Java only.
SparkContext	Entry point for core Spark. Creates RDDs, accumulators, and broadcast variables.
SparkSession	Unified entry point (since Spark 2.0). Wraps SparkContext + SQLContext + HiveContext.
Transformation	Lazy operations that define a new RDD/DF (map, filter, groupBy, join). Not executed immediately.
Action	Triggers actual computation (collect, count, show, save). Executes the DAG.
Lazy Evaluation	Transformations are NOT executed until an action is called. Allows Spark to optimise the plan.
Partition	Unit of parallelism. Each partition processed by one task on one executor.
DAG Scheduler	Breaks job into stages. Stages broken into tasks sent to executors.

PySpark — Word Count

```
from pyspark.sql import SparkSession spark =
SparkSession.builder.appName('WordCount').getOrCreate() sc = spark.sparkContext # Read
file into RDD lines = sc.textFile('hdfs:///input/text.txt') # MapReduce pipeline counts =
(lines .flatMap(lambda line: line.split()) .map(lambda word: (word.lower(), 1))
.reduceByKey(lambda a, b: a + b) .sortBy(lambda x: x[1], ascending=False)) # Action –
triggers execution counts.saveAsTextFile('hdfs:///output/wordcount')
print(counts.take(10))
```

Spark Ecosystem Libraries

Library	Purpose	Key APIs
Spark SQL	SQL + structured data	DataFrame, Dataset, SQL queries
Spark Streaming	Real-time micro-batch processing	DStream, window(), updateStateByKey()
Structured Streaming	Streaming on DataFrames	readStream, writeStream, watermark
MLlib	Distributed machine learning	Pipeline, Transformer, Estimator
GraphX	Graph-parallel computation	Graph, VertexRDD, EdgeRDD, Pregel

9. Quick Comparison — All Tools

Tool	Type	Language	Best For	Underlying Engine
HDFS	Storage	Java API / Shell	Reliable distributed storage	—
MapReduce	Processing	Java/Python	Batch ETL, aggregations	YARN
HBase	NoSQL DB	Java / Shell	Random R/W on HDFS	HDFS
Hive	SQL Warehouse	HiveQL (SQL)	OLAP, reports, analytics	MapReduce / Tez
Pig	Dataflow Script	Pig Latin	ETL pipelines, log proc.	MapReduce
Spark	Unified Engine	Scala/Python/Java	Batch+Stream+ML+Graph	Standalone/YARN/K8s
ZooKeeper	Coordination	Java	Distributed coordination	—
Sqoop	Ingestion	CLI	RDBMS <-> HDFS transfer	MapReduce
Flume	Ingestion	Config-based	Log collection to HDFS	—
Kafka	Messaging	Java / Client libs	Real-time event streaming	—

Viva Questions — Big Data & Memory-Efficient DS

Q1. What are the 5 V's of Big Data?

- Volume (size), Velocity (speed), Variety (types), Veracity (quality), Value (usefulness).

Q2. What is a Bloom Filter and where is it used?

- A probabilistic data structure for set membership testing. Uses k hash functions and a bit array. Has false positives but never false negatives. Used in Cassandra, HBase (to skip SSTable lookups), CDNs, web caches, Google BigTable.

Q3. What is HyperLogLog?

- A probabilistic algorithm for estimating the cardinality (number of unique elements) of a set. Uses ~1.5 KB of memory to estimate billions of unique items with ~2% error. Used in Redis PFCOUNT, Google Analytics.

Q4. What is a Count-Min Sketch?

- A probabilistic structure to estimate the frequency of events in a data stream. Uses a 2D array of counters and multiple hash functions. May overcount (false positives) but never undercount.

Q5. What is an LSM Tree? Where is it used?

- Log-Structured Merge Tree. Data written sequentially to in-memory MemTable, then flushed to immutable SSTables on disk. Compaction merges SSTables. Used in HBase, Cassandra, LevelDB, RocksDB for write-heavy workloads.

Q6. Difference between exact and approximate data structures?

- Exact structures (HashMap, TreeSet) give precise answers but require $O(n)$ space. Approximate structures (Bloom Filter, HLL, CMS) sacrifice exactness for dramatically lower memory — important when n is billions.

Viva Questions — Hadoop & HDFS

Q1. What is HDFS? How is it different from normal file systems?

- HDFS is a distributed file system designed for commodity hardware. Unlike local file systems, it splits large files into blocks (128 MB default), replicates each block 3 times across different DataNodes, and provides fault tolerance automatically.

Q2. What is the role of NameNode?

- NameNode is the master of HDFS. It stores filesystem metadata (file-to-block mapping, block-to-DataNode mapping, permissions). It does NOT store actual data. It's a Single Point of Failure (SPOF) unless HA (High Availability) is configured with a standby NameNode.

Q3. What is Secondary NameNode? Is it a backup NameNode?

- No. The Secondary NameNode is NOT a backup. It periodically merges the FSImage and EditLog to create a new FSImage, preventing the EditLog from growing too large. Data recovery still requires the primary NameNode.

Q4. What is rack awareness in HDFS?

- HDFS is aware of the physical rack topology of the cluster. For replication factor 3, it places: 1st replica on local node, 2nd replica on a different node in the same rack, 3rd on a different rack. This balances reliability (cross-rack) with write performance (intra-rack).

Q5. What happens when a DataNode fails?

- The NameNode detects the failure when it stops receiving heartbeats (after 10 minutes by default). It then initiates re-replication of the affected blocks from other DataNodes to maintain the replication factor.

Q6. What is the difference between -put and -copyFromLocal?

- Both upload files from local filesystem to HDFS. -put is a newer alias. -copyFromLocal is semantically identical. -moveFromLocal also uploads but deletes the local source file.

Q7. What is YARN and what are its components?

- YARN (Yet Another Resource Negotiator) is Hadoop's resource management layer. Components: ResourceManager (master, allocates resources cluster-wide), NodeManager (per-node, manages containers), ApplicationMaster (per-job, negotiates resources), Container (unit of resource allocation with CPU + RAM).

Viva Questions — MapReduce

Q1. Explain the phases of MapReduce with an example.

- Using WordCount: Input Splits → Mappers read lines and emit (word, 1) pairs → Shuffle/Sort groups all pairs by word key → Reducers sum values for each word → Output written to HDFS.

Q2. What is a Combiner? Why is it used?

- A Combiner is a local mini-Reducer that runs on each Mapper's output before the shuffle. It aggregates locally to reduce data transferred over the network. For WordCount, Combiner sums counts locally. It can only be used when the operation is commutative and associative.

Q3. What is the difference between map-side join and reduce-side join?

- Reduce-side join: both datasets sent to Reducers where join happens. Handles any size but expensive shuffle. Map-side join: smaller dataset loaded into Mapper's memory (DistributedCache). Join happens in Mapper. No Reducer needed. Much faster when one dataset fits in RAM.

Q4. What is a Partitioner in MapReduce?

- Partitioner determines which Reducer receives which intermediate key. Default HashPartitioner: reducer = (key.hashCode & Integer.MAX_VALUE) % numReducers. Custom Partitioners can ensure specific keys go to specific Reducers (e.g. range partitioning for sorted output).

Q5. How do you implement a Natural Join using MapReduce?

- Tag each record with its source table (e.g. emit (join_key, ('TableA', rest_of_row))). Reducer receives all records for a given join_key from both tables and cross-joins TableA records with TableB records having the same key.

Q6. What is speculative execution in Hadoop?

- If a task is running slower than average (possible straggler), Hadoop launches a duplicate copy on another node. Whichever finishes first wins; the other is killed. Improves overall job completion time at the cost of extra resource usage.

Q7. What are InputFormat and OutputFormat?

- InputFormat defines how input files are split and read. TextInputFormat reads line-by-line. SequenceFileInputFormat reads binary. KeyValueTextInputFormat splits on a delimiter. OutputFormat mirrors this for output. Custom formats can be written for special file types.

Viva Questions — Apache HBase

Q1. What is HBase and how does it differ from RDBMS?

- HBase is a distributed, column-oriented NoSQL database on HDFS. Unlike RDBMS: no fixed schema, no SQL joins, no transactions across rows, no referential integrity. Optimised for sparse data with billions of rows. Scales horizontally across commodity hardware.

Q2. What is a Column Family in HBase?

- A Column Family is a group of columns stored together physically on disk (same HFile). Must be defined at table creation. Each family can have unlimited column qualifiers added dynamically. Best practice: limit to 1-3 column families per table for performance.

Q3. What is the role of ZooKeeper in HBase?

- ZooKeeper provides distributed coordination. It: tracks which RegionServer is the active Master, stores the ROOT/META table location so clients can find regions, handles failure detection via session timeouts, and provides distributed locks.

Q4. Explain the HBase write path (MemStore + WAL + HFile).

- 1) Client sends Put. 2) Appended to WAL (Write-Ahead Log) for durability. 3) Stored in MemStore (in-memory sorted buffer) in RegionServer. 4) When MemStore threshold reached, flushed to HFile on HDFS. 5) Compaction merges small HFiles into larger ones periodically.

Q5. What is a Region in HBase?

- A Region is a contiguous sorted range of rows. Tables are auto-split into Regions as they grow. Each Region is served by exactly one RegionServer. Region splitting is triggered when size exceeds `hbase.hregion.max.filesize` (default 10 GB).

Q6. What is the difference between Get and Scan in HBase?

- Get retrieves a single row by RowKey in $O(1)$. Scan iterates over a range of rows. Scan supports start/stop row, filters, and column family/qualifier restrictions. Full table scans are expensive; always use RowKey prefixes and filters to limit scan range.

Q7. Why should RowKey design be chosen carefully in HBase?

- RowKeys are sorted lexicographically. Bad RowKeys (e.g. monotonically increasing timestamps) cause 'hot-spotting' — all writes go to one Region. Solutions: reverse timestamp (most recent first), salting (add hash prefix), or composite keys (userId + timestamp). Good RowKey design is critical for both write and read performance.

Viva Questions — Apache Hive

Q1. What is Hive Metastore?

- The Metastore is a relational database (Derby for local, MySQL/PostgreSQL for production) that stores Hive's metadata: table names, schemas, column types, partitions, and HDFS locations. Without the Metastore, Hive has no knowledge of tables.

Q2. What is partitioning in Hive and why is it useful?

- Partitioning physically separates data into sub-directories by partition column value (e.g. /data/year=2024/month=01/). When querying with a WHERE clause on the partition column, Hive reads only relevant directories — dramatically reducing I/O and query time.

Q3. What is bucketing in Hive?

- Bucketing divides data within a partition into a fixed number of files (buckets) based on the hash of a column. Useful for efficient sampling and improving join performance when both tables are bucketed on the join key (bucket map join).

Q4. What file formats does Hive support? Which is most efficient?

- TextFile (default, human-readable), SequenceFile (binary key-value), ORC (Optimized Row Columnar), Parquet (columnar), Avro (schema evolution). ORC and Parquet are most efficient for analytics — columnar storage, compression, predicate pushdown.

Q5. What is schema-on-read vs schema-on-write?

- Schema-on-write (RDBMS): schema enforced when data is inserted — reject bad data. Schema-on-read (Hive): data stored as-is; schema applied only when queried. Pros: flexible ingestion. Cons: query may fail on bad data.

Q6. Difference between ORDER BY, SORT BY, DISTRIBUTE BY, CLUSTER BY in Hive?

- ORDER BY: global sort, single Reducer, slow. SORT BY: local sort per Reducer, not globally sorted. DISTRIBUTE BY: controls which Reducer each row goes to (like Partitioner). CLUSTER BY = DISTRIBUTE BY + SORT BY on same column.

Q7. What is a UDF in Hive?

- User-Defined Function. Three types: UDF (one input row → one output), UDAF (multiple rows → one aggregated output, like AVG), UDTF (one row → multiple output rows, like EXPLODE). Written in Java and registered with CREATE FUNCTION.

Viva Questions — Apache Pig

Q1. What is Pig Latin and how does it compare to SQL?

- Pig Latin is a dataflow language — you describe transformations step by step (procedural). SQL is declarative — you describe what result you want, not how. Pig gives more control over execution order and is better for complex multi-step ETL pipelines.

Q2. What is lazy evaluation in Pig?

- Like Spark, Pig does not execute transformations immediately. Execution is deferred until a DUMP or STORE action is called. This allows the Pig optimizer to reorder and combine steps into an efficient MapReduce plan.

Q3. What is the difference between FOREACH...GENERATE and FILTER?

- FILTER removes rows based on a boolean condition (row selection). FOREACH...GENERATE transforms columns within each row (projection and computation). FILTER = SQL WHERE; FOREACH...GENERATE = SQL SELECT.

Q4. How does Pig handle schema?

- Pig is schema-optional. You can define a schema with AS clause in LOAD, or process data without schema. Without schema, fields accessed by positional notation (\$0, \$1, ...). DESCRIBE shows the schema of a relation.

Q5. What is GROUP ALL vs GROUP BY in Pig?

- GROUP BY groups rows by a specific column (like SQL GROUP BY). GROUP ALL puts all rows in a single group — useful for computing global aggregates like total count or global average across the entire dataset.

Viva Questions — Apache Spark

Q1. Why is Spark faster than MapReduce?

- Spark keeps intermediate results in RAM (RDD caching) instead of writing to HDFS between stages. It also uses a DAG execution engine that optimises the full computation graph before running, and pipelines stages within the same node to minimise data movement.

Q2. What is an RDD and its properties?

- RDD (Resilient Distributed Dataset) is Spark's core abstraction. Properties: Immutable (can't be changed, only transformed), Distributed (partitioned across nodes), Resilient (fault-tolerant via lineage — lost partitions recomputed from lineage graph), Lazy (transformations not executed until action called).

Q3. Difference between transformations and actions in Spark?

- Transformations (map, filter, flatMap, groupByKey, join) are lazy — return a new RDD/DF without executing. Actions (collect, count, take, save, reduce) trigger DAG execution. Every time an action is called, the entire lineage is re-executed unless RDDs are cached.

Q4. What is the difference between map and flatMap?

- map applies a function to each element and returns one result per element (1-to-1). flatMap applies a function that returns an iterable and flattens all results into one collection (1-to-many). For WordCount, flatMap splits each line into words.

Q5. What is the difference between groupByKey and reduceByKey?

- groupByKey sends ALL values for each key to Reducer — causes massive shuffle. reduceByKey performs local pre-aggregation on each partition (like Combiner) before shuffle. Always prefer reduceByKey/aggregateByKey over groupByKey for aggregations.

Q6. What is DAG in Spark?

- DAG (Directed Acyclic Graph) is Spark's execution model. Each action creates a new Job. The Job is split into Stages at shuffle boundaries. Stages are split into Tasks (one per partition). DAGScheduler optimises stages; TaskScheduler assigns tasks to executors.

Q7. What is caching/persistence in Spark? What levels exist?

- cache() = persist(MEMORY_ONLY). persist() allows choosing storage level: MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_SER (serialized). Caching is critical for iterative algorithms (ML) to avoid recomputing lineage on each iteration.

Q8. What is a broadcast variable?

- A read-only variable cached on every executor rather than sent with every task. Useful for broadcasting a small lookup table to all Mappers to implement a broadcast join without shuffling the large table. Created with sc.broadcast(dict).

Q9. Explain Structured Streaming in Spark.

- Structured Streaming treats a live data stream as an unbounded DataFrame. It uses the same DataFrame/SQL API as batch processing. Supports event-time processing, watermarking for late data, and output modes: complete (full table), append (new rows only), update (changed rows only).

Viva Questions — Cross-Topic & Tricky Questions

Q1. Hive vs HBase — when would you use each?

- Use Hive for analytical queries (OLAP) on large historical datasets where you write SQL and don't need real-time access. Use HBase for operational workloads (OLTP-like) where you need millisecond random reads/writes by RowKey — e.g. user profile lookups, time-series data, IoT sensor data.

Q2. Pig vs Hive — when would you use each?

- Use Hive when your team knows SQL and you need ad-hoc analytics. Use Pig when you need a complex multi-step ETL pipeline with custom transformations — Pig Latin is more flexible for non-tabular, messy data.

Q3. Can Spark replace Hadoop MapReduce entirely?

- Yes for processing — Spark is a superset of MapReduce functionality and much faster. But Spark still uses HDFS (or S3/GCS) for storage. YARN can also manage Spark resources. Many modern clusters use HDFS + YARN + Spark, with MapReduce fully replaced.

Q4. What is data locality in Hadoop?

- Data locality = moving computation to where data lives rather than moving data to computation. HDFS-aware schedulers prefer to run Mapper tasks on the same node that holds the data block. Node-local > Rack-local > Any node. Reduces network bandwidth dramatically.

Q5. What is skew in MapReduce/Spark and how do you handle it?

- Data skew = some keys have vastly more records than others, causing some tasks to run much longer. Solutions: salting (add random prefix to keys, then aggregate twice), repartitioning with more partitions, using aggregateByKey instead of groupByKey, or broadcasting smaller dataset for joins.

Q6. What is idempotency and why is it important in distributed systems?

- An operation is idempotent if applying it multiple times gives the same result as applying once. Critical in MapReduce/Spark because tasks can be re-executed on failure. If output writes are not idempotent, retries could corrupt results. Hadoop uses output committers to handle this atomically.

Q7. What is the CAP theorem? Where do HBase and Hive fall?

- CAP: a distributed system can guarantee at most 2 of: Consistency, Availability, Partition Tolerance. HBase: CP (Consistent + Partition Tolerant — sacrifices availability during network partitions). Hive: not a real-time DB, so CAP doesn't strictly apply — it's a batch analytics tool.

10. Last-Minute Cheat Sheet — Numbers & Facts to Memorise

Fact	Value / Answer
Default HDFS block size	128 MB (was 64 MB in older versions)
Default HDFS replication factor	3
NameNode heartbeat from DataNode	Every 3 seconds
DataNode block report interval	Every 1 hour (3600 seconds)
DataNode considered dead after	10 minutes of missed heartbeats
HBase default MemStore flush size	64 MB
HBase default max region size	10 GB (triggers auto-split)
Spark cache() = persist level	MEMORY_ONLY
Spark default parallelism	Number of cores in cluster
MapReduce: map output key sort	Happens in Shuffle/Sort phase automatically
Bloom Filter error type	False positives only (never false negatives)
HyperLogLog typical error	~2% with 1.5 KB memory
YARN containers	ResourceManager allocates; NodeManager executes
Hive default execution engine	MapReduce (change to Tez/Spark for speed)
Pig execution modes	Local (single JVM) and MapReduce (cluster)
Spark vs MapReduce speed	Up to 100x faster in-memory, 10x on disk
HBase data model unit	Cell = RowKey + ColumnFamily + Qualifier + Timestamp + Value
ZooKeeper port	2181
HDFS NameNode web UI port	9870 (Hadoop 3.x), 50070 (Hadoop 2.x)
YARN ResourceManager web UI	8088

Pro Tips for Viva: When asked 'Why Spark over MapReduce?' — say in-memory DAG execution, not just 'Spark is faster'. When asked about HBase RowKey design — mention hot-spotting and salting. When asked about HDFS fault tolerance — mention replication AND rack awareness. For any 'difference between X and Y' — give use cases, not just definitions.